RelayAgent: Mobile Task Automation by Delegating Subtasks to In-App AI Assistants

Jingsheng Yan*, Fangnuo Wu*, Mingkai Dong, Haibo Chen

Shanghai Jiao Tong University

*Equal contribution {yanjingsheng, wufangnuo, mingkaidong, haibochen}@sjtu.edu.cn

Abstract

Mobile agents usually automate applications either through explicit vendor APIs or through low-level GUI interactions. This paper explores a third substrate: in-app AI assistants that already have access to application-specific context, domain logic, and task workflows. We present RelayAgent, a new mobile agent architecture that decomposes user requests into subtasks, delegates those subtasks to suitable in-app assistants by issuing natural language requests, and orchestrates the returned results across applications. The key challenge is deciding which assistant can reliably handle which subtask, because assistant capabilities are implicit, uneven, and not exposed as well-defined APIs. RelayAgent addresses this challenge with a dynamic tool library that discovers, validates, and refines assistant-level capabilities from execution experience, together with reusable entry scripts that reduce repeated GUI overhead. Our evaluation shows that compared to a GUI agent, this delegation model reduces end-to-end task completion time and LLM token consumption, and achieves a higher success rate—tasks on which a GUI agent fails due to interaction complexity can be handled by delegating to in-app assistants.

1 Introduction

Mobile agents (Hu et al., 2025; Zhang et al., 2025; Wang et al., 2024; Wen et al., 2024) aim to automate user tasks by interacting with smartphone applications on behalf of the user. For example, a user may ask the agent to "find a highly rated nearby restaurant and navigate there," and the agent coordinates the necessary applications to complete the task and deliver the result without requiring the user to operate any application manually. Such potential to reduce user effort has driven growing research interest in mobile agents (Liu et al., 2025; Zhang et al., 2024a).

Existing mobile agent systems usually pursue one of two routes. The first route asks applications to publish APIs for agents to call, such as Model Context Protocol (MCP) (Anthropic, 2024), Google Agent2Agent (A2A) (Google, 2025a), Apple App Intents (Apple, 2022) or Android App Functions (Google, 2026a). This route can be efficient and reliable when the interface exists, but coverage is inevitably limited: an agent can only invoke a capability after the application has explicitly published an API for it. Many third-party applications either expose no such interface or expose only a narrow subset of what their interactive UI can do (Zhang et al., 2025; Wang et al., 2024; Wen et al., 2024; Liu et al., 2025). The second route is GUI agents, which emulate human interactions by observing the screen and issuing actions such as taps, scrolls, and text entry, making them applicable to any application without prior API support (Zhang et al., 2025; Wang et al., 2024; Wen et al., 2024; Cheng et al., 2024). However, this generality comes at a high cost: every action requires an LLM call to interpret the current screen state, making execution slow and token-intensive; GUI grounding errors further cause incorrect actions that

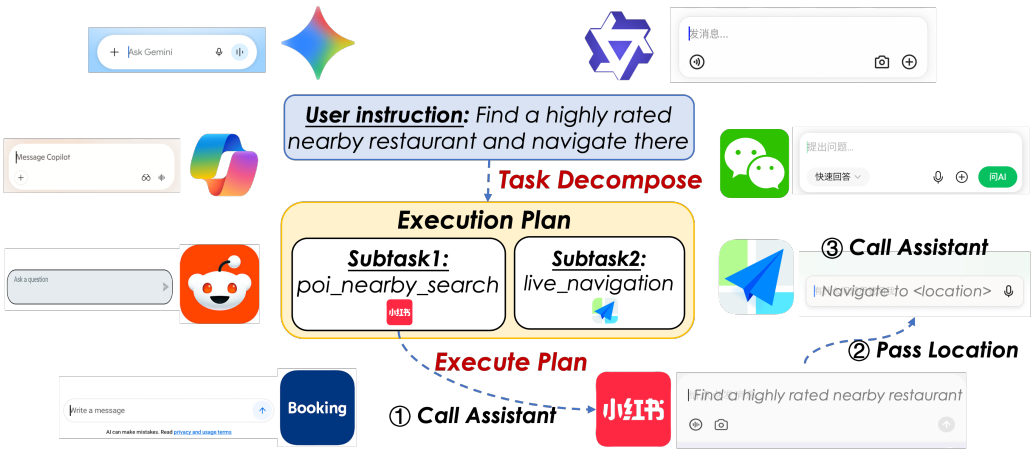

Figure 1: Architecture of delegation-based agent.

require retries, which compounds latency and leads to cascading failures on long-horizon tasks (Hu et al., 2025; Rawles et al., 2025).

This paper starts from a different observation: many popular applications have already embedded their own AI assistants. These assistants are not merely decorative chat boxes, but often have privileged access to the application's domain data, ranking logic, and action flows, and can complete application-local tasks through natural language. This trend spans application categories and markets. For instance, in map services, Google Maps is adding Ask Maps powered by Gemini (Google, 2026b); in shopping, Alibaba's Taobao exposes a Qwen-powered shopping assistant (Alibaba Group, 2026); in travel, Trip.com offers TripGenie (Trip.com, 2023), while Booking.com has introduced AI Trip Support and related agentic AI features (Booking.com, 2025a); in messaging and social applications, Reddit's AI search (Reddit, 2024) and Rednote's Diandian (Shao & Jiang, 2025) bring assistant-style interaction into existing user communities; and in productivity suites, assistants such as Microsoft Copilot (Microsoft, 2023) and WPS AI (WPS Office, 2023) are embedded.

These in-app assistants open a *third substrate* for mobile agents. A mobile agent need not wait for vendors to publish APIs, nor issue low-level UI actions one by one as GUI agents do. Instead, a mobile agent can act as an orchestrator: it decomposes the user request into application-local subtasks and delegates each subtask to the corresponding in-app assistant via natural language instructions, which leverages its privileged access to domain data and task workflows to complete the subtask internally and return the result.

In this paper, we propose RelayAgent, a new mobile agent architecture that explores coordinating in-app assistants for mobile task automation. Given a user request, RelayAgent decomposes the request into application-local subtasks, selects an application whose assistant is likely to handle each subtask, sends a natural-language instruction to that assistant, and extracts the structured result needed by the next step. For the example task in Figure 1 (i.e., "find a highly rated nearby restaurant and navigate there"), RelayAgent can first ask a content-discovery assistant to find the best nearby restaurant, obtain the selected restaurant information, and then ask a map assistant to start directions to that restaurant. Rather than completing every application-local operation itself, RelayAgent delegates such work to the application's own assistant and focuses on cross-application planning, result passing, and fallback.

Delegating to in-app assistants offers advantages over both API and GUI agents. Compared with API agents (Song et al., 2025; Du et al., 2024), it provides richer coverage: in-app assistants are more prevalent than structured APIs, and often expose more functionality. Compared with GUI agents (Zhang et al., 2024b; Lu et al., 2024; Zhang et al., 2025; Wang et al., 2024), delegation reduces both end-to-end completion time and LLM token cost: a multi-step in-app operation collapses into a single natural-

language instruction, eliminating the per-action screenshot, inference, and click loop. When no suitable assistant exists, RelayAgent falls back to GUI control, so coverage is never worse than a pure GUI agent.

The central challenge for making RelayAgent efficient and reliable is to decide which subtasks can be delegated to which in-app assistants. This decision is difficult because these assistants expose natural-language surfaces rather than well-defined APIs, leaving their capabilities implicit and uneven. Before delegation, the agent must know what each assistant can and cannot do: e.g., whether a map assistant can return ranked restaurants, whether a shopping assistant can buy products under constraints. Without this knowledge, the agent wastes time sending impossible requests or misroutes subtasks, delaying the fallback to GUI agents.

RelayAgent addresses this problem by dynamically modeling in-app assistants as a tool library. For each application, RelayAgent maintains a set of tool templates that describe assistant-level capabilities, such as "find restaurants near *location* under *constraints*" or "navigate from *source* to *destination*." These assistant-level tools differ from the tools used by existing API-agent and GUI-agent approaches. Unlike API-agent tools, they are not predefined by vendors, but are discovered, validated, and refined at runtime from prior delegation attempts. Unlike GUI-agent tools, they are not low-level actions such as clicks, but summarize a higher-level application capability. This library lets RelayAgent match subtasks to known-capable assistants, fall back to a GUI agent when a capability is known to be unsupported, and explore assistants' capabilities for uncertain tasks, updating the library based on the outcome.

RelayAgent further improves efficiency by recording the execution script of reaching each application's assistant. The repeated procedure of locating the in-app assistant search box, focusing it, entering the instruction, and submitting the request is mostly a device- and application-specific routine. Rather than ask a GUI agent to rediscover this routine on every task, RelayAgent turns successful interaction traces into reusable scripts.

In the evaluation, we show that RelayAgent can complete complex tasks with 1.4–2× (average 1.8×) lower end-to-end completion time and 7–10× (average 8.8×) lower LLM token usage than the baseline GUI agent (general_e2e agent in MobileWorld (Tongyi-MAI, 2025a; Kong et al., 2025)). In addition, RelayAgent can finish 46%, 49%, and 83% of tasks in AndroidDaily (StepFun AI, 2025; Sui et al., 2026), MobileWorld (Tongyi-MAI, 2025a; Kong et al., 2025), and our proposed RelayBench benchmarks, outperforming the baseline GUI agent's 31%, 34%, and 77% success rates.

This paper makes the following contributions:

- We identify in-app AI assistants as a practical third substrate for mobile task automation, complementary to API-based and GUI-based mobile agents.
- We present RelayAgent, a mobile agent architecture that decomposes user tasks and delegates subtasks to in-app assistants through natural language.
- We design a dynamic assistant-tool library that learns and maintains the capability boundaries of in-app assistants from execution experience.

2 In-App AI Assistants Are Prevalent

2.1 Characterizing In-App AI Assistants

In-app AI assistants refer to AI-powered assistants built directly into applications to help users accomplish tasks and navigate application-specific features through natural language. As shown in Figure 1, they are often presented as a search or chat interface where users can enter requests such as "Buy me a cup of coffee" or "Book a flight to Hong Kong" in natural language, and the assistant will complete the task and return the result.

Table 1: In-app AI assistant capability coverage across representative applications. ✓ indicates that the application's AI assistant has the corresponding capability.

Capability	**Gemini** (Google, 2025c)	**Reddit** (Reddit, 2024)	**Booking** (Booking.com, 2025a)	**Copilot** (Microsoft, 2023)	**Walmart** (Walmart, 2025)	**WPS Office** (WPS Office, 2023)	**Qwen** (Alibaba Group, 2025b)	**Amap** (Alibaba Group, 2025a)	**WeChat** (South China Morning Post, 2025)	**Rednote** (Shao & Jiang, 2025)	**Ctrip** (Ctrip, 2025)	**Qunar** (TravelDaily, 2023)	**Zhihu** (South China Morning Post, 2024)
General AI													
Foundation LLM (QA, Translation, ...)	✓			✓			✓						
Location & Transport													
POI search	✓		✓	✓			✓	✓	✓	✓	✓	✓	
Route planning (Distance & ETA)	✓						✓	✓					
Navigation	✓						✓	✓					
Ride-Hailing							✓	✓					
Travel & Booking													
Trip Planning			✓								✓	✓	
Train Search & Booking							✓				✓	✓	
Flight Search & Booking							✓				✓	✓	
Hotel Search & Booking			✓				✓				✓	✓	
Event Tickets Search & Booking							✓						
Shopping & Commerce													
Product Search & Recommendations				✓	✓		✓						
Food Ordering							✓						
Order Tracking							✓						
Personal Information Management													
Calendar Access	✓												
Calendar Management	✓												
Alarm & Reminder Setting	✓												
SMS Access	✓												
SMS Sending	✓												
Email Access	✓												
Email Composition & Sending	✓												
Content Creation													
Docs & Slides Generation	✓					✓							
Notes & Tasks													
Notes Access	✓												
Notes Management	✓												
Tasks Access	✓												
Tasks Management	✓												
Social & Content Search													
In-App Content Search		✓							✓	✓			✓
System													
System Settings Management	✓												

An increasing number of mainstream applications have adopted in-app AI assistants, enabling users to access and perform a broad range of application functions through a unified conversational experience. For example, 15 out of 32 commonly used Chinese applications involved in the AndroidDaily (StepFun AI, 2025; Sui et al., 2026) benchmark have adopted in-app AI assistants. Besides, some applications are themselves general-purpose AI assistants (e.g., Gemini (Google, 2025c) and Qwen (Alibaba Group, 2025b)) that can conduct tasks like hailing rides and managing calendars, all within a single conversational interface.

We summarize several representative global and Chinese applications that support in-app AI assistants and their capabilities in Table 1. We expect the number and capability of in-app AI assistants to keep growing: as large models become cheaper to serve, embedding a capable assistant is becoming the default pattern for consumer applications.

2.2 A New Route: Delegation-based Mobile Agent

We propose a new route that sits between API agents and GUI agents: delegating application-local subtasks to in-app AI assistants. In this model, the outer agent is responsible only for cross-application planning and decomposing the user request into subtasks that individual applications can handle. For each subtask, the agent performs only a few fixed GUI actions—navigating to the target application, typing a natural-language instruction into its assistant interface, and submitting it—while the assistant itself handles all application-internal operations and returns the result. This design combines the strengths of the two existing routes.

Richer coverage than API agents. Very few applications expose APIs like MCP (Anthropic, 2024) or A2A (Google, 2025a), while in-app AI assistants are increasingly prevalent. Among the applications in Table 1, only Amap provides a publicly accessible MCP interface (Amap, 2025); Booking.com also offers an MCP server, but only to registered affiliate partners (Booking.com, 2025b). Moreover, even when an application does publish an API, its in-app assistant typically exposes more functionality than the API does. For example, Amap's MCP interface does not support flight ticket queries, yet its in-app AI assistant handles them natively. We conjecture this is because an in-app assistant improves user experience and keeps traffic inside the application, whereas a public API does the opposite—third parties can access the same functionality without users ever opening the app.

Lower cost and faster completion time than GUI agents. For every action, GUI agents must call the LLM to interpret the current screen and decide the next step, which is time-consuming and costly. In a delegation-based agent, the LLM is called only during the initial planning step and for parsing inter-subtask results; all application-internal steps are executed by the in-app assistant without further LLM involvement. This reduces both token consumption and end-to-end latency significantly, especially for subtasks that would otherwise require many clicks and page navigations deep inside an application.

3 Design of RelayAgent

3.1 Overview of RelayAgent

Figure 3 illustrates how RelayAgent completes the user's instruction "navigate to Hong Kong International Airport" step by step, and compares its workflow with that of a GUI-based agent (shown in Figure 2).

In general, a GUI agent invokes the LLM at every interaction step. In each step, it sends the user's instruction together with the current screenshot to the model, which then predicts the next action (e.g., clicking a button or entering text). This process is repeated until the task is completed.

In contrast, RelayAgent invokes the LLM only during the initial planning step and for parsing inter-

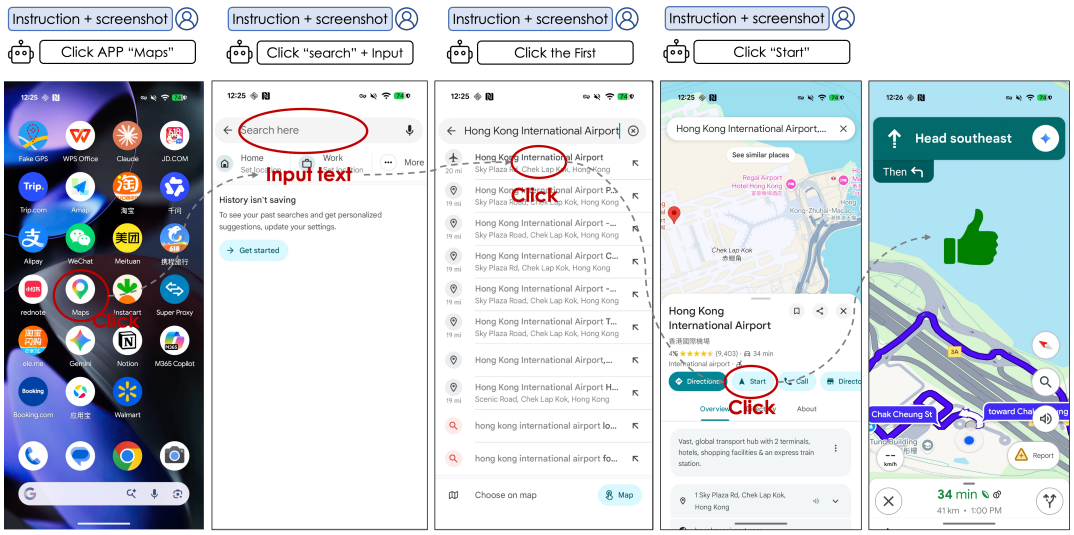

Figure 2: GUI Agent. GUI agent uses multiple rounds of LLM calls to decide the next action (e.g., input text, click button).

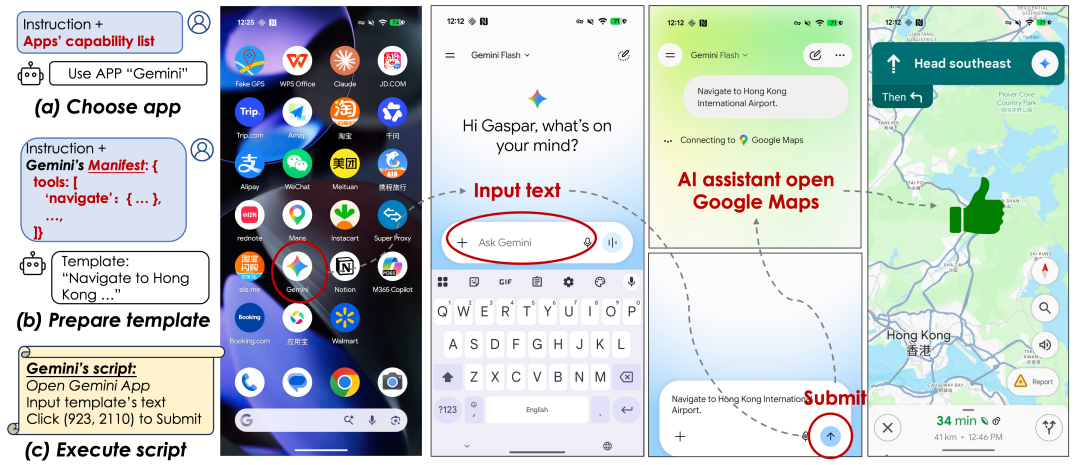

Figure 3: RelayAgent. RelayAgent completes the user instruction by (a) decomposing it into one subtask, selecting the appropriate application, i.e., Gemini; (b) generating an execution plan together with the natural-language request for Gemini's AI assistant; (c) executing the subtask via pre-recorded entry scripts.

subtask results. The LLM is used to determine the appropriate application, generate the natural-language request for the in-app assistant, and extract structured results returned between subtasks. All application-internal steps are carried out by deterministic scripts, avoiding the repeated perception-and-action loop required by GUI-based agents and substantially reducing LLM inference overhead.

3.2 Detailed Procedure of RelayAgent

3.2.1 Initial planning step.

The initial planning step decomposes the user instruction into subtasks and produces an execution plan. Note that the example in Figure 3 contains only one subtask for simplicity; in practice the LLM may produce several subtasks. To avoid passing the full capability details of every application to the LLM at once, planning proceeds in two steps.

Step (a) selects the application(s) that can handle the user instruction. RelayAgent calls the LLM with

the user instruction and a compact *application capability list*, which is a high-level summary of what each application's in-app assistant can do. The LLM decomposes the instruction into one or several subtasks and assigns each to an application; subtasks that no assistant can handle are marked to fall back to a conventional GUI agent.

Step (b) formulates the execution plan. RelayAgent retrieves the *manifest* of each selected application (maintained as a local JSON file), which enumerates the assistant's tools as a set of parameterized templates with constraint descriptions. For example, Gemini's navigation tool contains templates such as "navigate to *destination* under *constraints*" (constraints can be walking, driving, or hiking, but not air or sea travel) and "lookup travel time from *source* to *destination* under *constraints*" (constraints can include a specific departure time), alongside other tools such as calendar read and email read. The LLM matches each subtask to a template, fills in the parameters to form a concrete natural-language request, and marks any subtask whose constraints fall outside the template's declared scope for GUI agent fallback.

3.2.2 Task execution step.

After getting an execution plan, RelayAgent executes the plan step by step.

The plan is composed of one or several subtasks. Given that each subtask is associated with one application, to execute a subtask, RelayAgent runs a pre-recorded entry script that navigates to the target application and submits the natural-language request to its in-app assistant. For example, Gemini's entry script consists of (1) open the Gemini app, (2) input the request text given by the execution plan[1], and (3) click the submit button at a fixed screen location to send the request. Each entry script is versioned per application release, as UI layouts may change across updates.

3.2.3 Inter-subtask result handoff.

After each subtask completes, RelayAgent extracts the result from the assistant's response. If a subsequent subtask depends on this result (e.g., passing a restaurant name to a navigation request), RelayAgent calls the LLM again with the updated context to generate the natural-language request for the next subtask.

3.3 Dynamic Tool Modeling

The *application capability list* and each application's *manifest* used during step (a) and (b) are both dynamically maintained and updated. This is because the true capabilities of each in-app assistant are not known in advance: RelayAgent discovers, validates, and refines them incrementally through execution experience.

Specifically, three types of updates can occur: (1) a new application and its capability summary are added to the capability list when the system encounters an application not previously known; (2) a new tool template is added to an application's manifest when a successful execution reveals a capability not yet recorded; (3) the constraint description of an existing template is refined when execution experience reveals that the constraint description is inaccurate or incomplete. The update is conducted after each execution of a subtask: RelayAgent will check whether the subtask is already correctly and completely described in the capability list and manifest. If not, RelayAgent will update the capability list or manifest accordingly.

4 Evaluation

In this section, we compare RelayAgent with a GUI agent on several benchmarks to answer the following questions:

[1]Gemini launches directly into its assistant interface upon opening, so no additional click is needed to reach the assistant; this is not the case for most other applications.

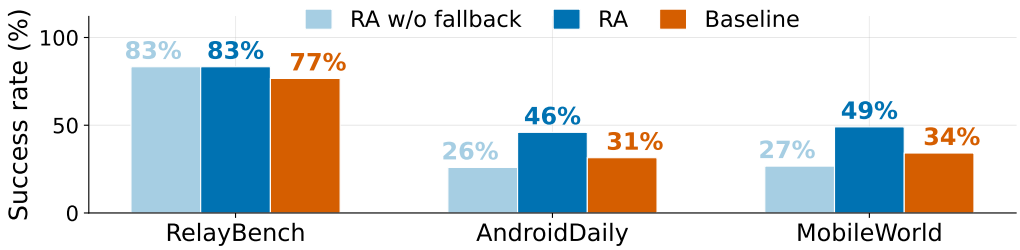

Figure 4: Success rate comparison. Success rate of our delegation-based agent with GUI fallback enabled (RA), the same agent without fallback (RA w/o fallback), and the baseline GUI agent.

- What is the **success rate** of RelayAgent? (§ 4.2)
- Does RelayAgent **complete tasks faster** compared to the GUI agent? (§ 4.3)
- Does RelayAgent **save LLM token consumption** compared to the GUI agent? (§ 4.4)

4.1 Experimental Setup

All experiments are conducted on the same physical device: a Pixel 9 (Google, 2024) mobile phone with 12 GB of memory, running Android 16 (Google, 2025b). We use the `general_e2e` configuration of MobileWorld (Tongyi-MAI, 2025a; Kong et al., 2025) as our baseline GUI agent, which takes the current screenshot as input and directly predicts the next action (tap, scroll, type, etc.), executing frame by frame in a loop. Both RelayAgent and the baseline share the same deployed Qwen3.5-27B (Qwen Team, 2026) model. We measure the real end-to-end completion time of each task, and then normalize the LLM inference portion across runs to remove serving-side variance: we pre-measure the model's prefill and decode throughput, and replace each call's measured inference time with $N_{\text{in}}/T_{\text{prefill}} + N_{\text{out}}/T_{\text{decode}}$, following the standard prefill/decode decomposition of LLM inference (Zhong et al., 2024), where N_{in} and N_{out} are the input and output token counts and T_{prefill}, T_{decode} are the measured throughputs. The reported end-to-end time is thus the measured non-LLM time plus the normalized LLM inference time.

We choose the following benchmarks:

- **MobileWorld** (Tongyi-MAI, 2025a; Kong et al., 2025): A benchmark of 201 tasks covering 20 applications (Mail, Clock, Settings, Chrome, etc.), emphasizing long-chain cross-app workflows.
- **AndroidDaily** (StepFun AI, 2025; Sui et al., 2026): A benchmark of 235 tasks covering 32 commonly used Chinese applications (WeChat, Alipay, Taobao, TikTok, etc.), emphasizing simple everyday tasks with broad application coverage.
- **RelayBench**: A benchmark of 30 tasks constructed from our own daily usage patterns, covering both global and Chinese applications. RelayBench tasks focus on applications not covered by the benchmarks above, and are substantially longer and involve more steps to reflect the complex multi-step workflows that arise in real daily use.

For MobileWorld, we exclude the 40 tasks that can only be completed through MCP-based tool calls and evaluate on the remaining 161 tasks. For all benchmarks, task success is determined by human evaluation: a task is considered successful only if both the trajectory and the final outcome are correct. Tasks in which the agent fails to terminate normally (e.g., entering an infinite action loop) are marked as failures. Each task is executed by both systems sequentially under identical device state; a cold-launch reset is performed between tasks to prevent state leakage from prior executions.

4.2 Success Rate

Figure 4 shows the success rate of RelayAgent and the baseline on each benchmark. RelayAgent achieves 46%, 49%, and 83% on AndroidDaily, MobileWorld, and RelayBench respectively, compared to 31%, 34%,

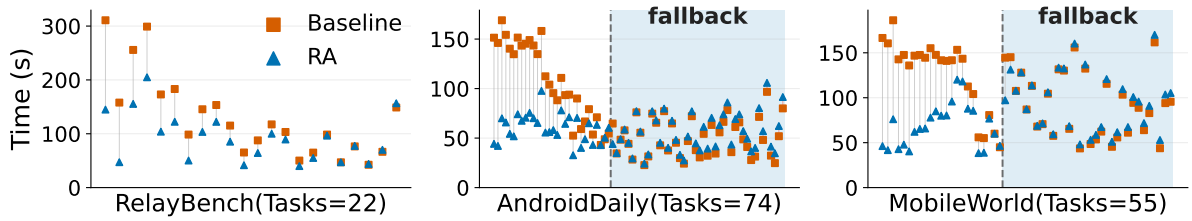

Figure 5: Task completion time. Each task is shown as a paired RelayAgent (blue) and baseline (orange) point at the same x-position, sorted by RelayAgent minus baseline. We only show tasks where both RelayAgent and the baseline succeed, and tasks that succeed with fallback to the GUI agent are grouped in the shaded region.

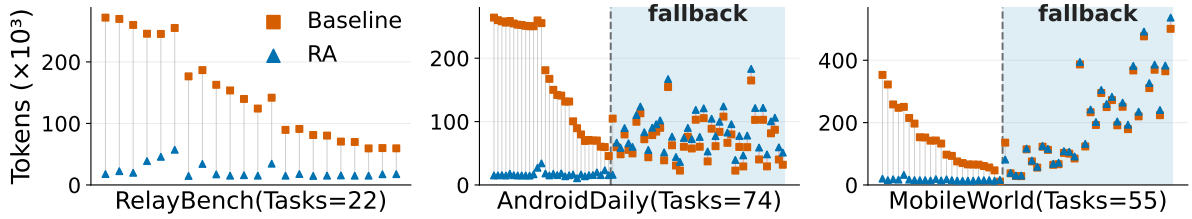

Figure 6: Task token consumption. Pairing and sorting follow Figure 5.

and 77% for the baseline GUI agent. Failures of RelayAgent without fallback are primarily caused by (1) the target application lacking an in-app AI assistant (e.g., 76% of MobileWorld failures stem from Mattermost (Mattermost, Inc., 2024), Mastodon (Mastodon gGmbH, 2024), and filesystem operations, which lack in-app AI assistants), or (2) the assistant's capabilities not covering the task's requirements (e.g., accounting for 31% of AndroidDaily failures).

RelayAgent has higher success rate than the baseline GUI agent on all benchmarks. This is because for tasks where the in-app assistant has the relevant capability, RelayAgent can complete them reliably, since the execution is handled entirely by the assistant rather than by a fragile sequence of GUI actions. We note that using a stronger backbone model can further raise GUI agent success rates—the best reported result on MobileWorld currently stands at 63.2% (Tongyi-MAI, 2025b)—but doing so also increases LLM token consumption. A stronger model would equally benefit RelayAgent by improving the success rate of its GUI fallback path.

4.3 End-to-End Completion Time

Figure 5 shows the task completion time of RelayAgent and the baseline GUI agent across three benchmarks; only tasks where both succeed are shown. Compared with the GUI agent baseline, RelayAgent achieves a 1.4–2× speedup (average 1.8×) when no fallback is needed across all benchmarks. The improvement mainly comes from the fact that RelayAgent replaces the baseline's loop of dozens of steps—each requiring a screenshot, VLM inference, and action prediction—with a single request to the in-app assistant. The gap is smaller on simpler tasks, because the GUI agent can complete them with only a few LLM calls and actions.

When fallback is needed, the additional overhead introduced by RelayAgent remains small (4 seconds on average, 7% of the total execution time). This is because our dynamic tool modeling accurately captures the capability boundaries of in-app assistants, avoiding useless delegation attempts.

4.4 LLM Token Consumption

Figure 6 shows the token consumption of RelayAgent and the baseline GUI agent, with the same configuration as Figure 5. The results show that when no fallback is needed, RelayAgent uses 7–10× (average 8.8×) fewer tokens than the baseline GUI agent across all benchmarks. This is because RelayAgent involves less LLM inference compared to the GUI agent. When fallback is needed, the additional token consumption remains small (10 K tokens on average, 5% of the total token usage).

5 Related Work

LLM tool use and dynamic tool discovery. Toolformer (Schick et al., 2023) and ToolLLM (Qin et al., 2024) demonstrate that LLMs can learn to invoke external APIs, but both assume that tool schemas are known and fixed in advance. RelayAgent faces a different challenge: in-app assistants do not expose predefined schemas, so capability boundaries must be discovered and refined incrementally from execution experience rather than specified upfront.

LLM task planning and delegation. HuggingGPT (Shen et al., 2023) uses an LLM controller to decompose tasks and dispatch subtasks to specialized AI models on Hugging Face, and AutoGen (Wu et al., 2023) provides a general framework for multi-agent collaboration through natural language conversation. Both approaches assume that the capabilities of each sub-agent are known and accessible via a clean programmatic interface. RelayAgent adopts a similar orchestration structure but targets in-app assistants whose capabilities are implicit and require discovery from execution experience.

6 Conclusion

This paper proposed RelayAgent, a mobile agent that delegates application-local subtasks to in-app AI assistants through natural language, limiting GUI interaction to the minimal steps needed to reach each assistant. A dynamic tool library refines assistant capabilities from execution experience to guide routing and fallback decisions. Evaluation shows that RelayAgent completes 46%, 49%, and 83% of tasks on AndroidDaily, MobileWorld, and RelayBench respectively, with substantially lower LLM token usage and end-to-end latency than a GUI-only baseline.

References

Alibaba Group. Alibaba Launches the World's First AI-native Map Application with Qwen, 2025a. URL https://www.alibabagroup.com/en-US/document-1889126073686294528.

Alibaba Group. Alibaba's Qwen App Surpasses 10 Million Downloads within the First Week of Public Beta Launch, 2025b. URL https://home.alibabagroup.com/document-1928528945544691712.

Alibaba Group. Alibaba Opens All of Taobao to Qwen AI, Ushering in a New Agentic Shopping Experience, 2026. URL https://www.alibabagroup.com/document-1991231293551017984.

Amap. Amap MCP Server, 2025. URL https://lbs.amap.com/api/mcp-server/summary.

Anthropic. Model Context Protocol Specification, 2024. URL https://modelcontextprotocol.io/specification/2024-11-05/basic.

Apple. App Intents, 2022. URL https://developer.apple.com/documentation/AppIntents/app-intents.

Booking.com. Booking.com Debuts Agentic AI Innovations, Adding to Its Robust Suite of GenAI Tools for Customers, 2025a. URL https://news.booking.com/bookingcom-debuts-agentic-ai-innovations-adding-to-its-robust-suite-of-genai-tools-for-customers/.

Booking.com. Booking.com Demand API MCP Server, 2025b. URL https://developers.booking.com/demand/docs/open-api/demand-api.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9313–9332, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.505. URL https://aclanthology.org/2024.acl-long.505/.

Ctrip. Ctrip's AI Trip Assistant Launches on Both App and Web, with One-Tap Sync for Smart Itinerary Planning, 2025. URL https://finance.sina.com.cn/jjxw/2025-09-16/doc-infqsnvi3651440.shtml.

Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls, 2024.

Google. Pixel 9, 2024. URL https://store.google.com/us/product/pixel_9_specs.

Google. A2A: A New Era of Agent Interoperability, 2025a. URL https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/.

Google. Android 16, 2025b. URL https://developer.android.com/about/versions/16.

Google. The Assistant Experience on Mobile Is Upgrading to Gemini, 2025c. URL https://blog.google/products-and-platforms/products/gemini/google-assistant-gemini-mobile/.

Google. App Functions, 2026a. URL https://developer.android.com/ai/appfunctions.

Google. Ask Maps and Immersive Navigation: New AI Features in Google Maps, 2026b. URL https://blog.google/products-and-platforms/products/maps/ask-maps-immersive-navigation/.

Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, Yuhuai Li, Shengze Xu, Shenzhi Wang, Xinchen Xu, Shuofei Qiao, Zhaokai Wang, Kun Kuang, Tieyong Zeng, Liang Wang, Jiwei Li, Yuchen Eleanor Jiang, Wangchunshu Zhou, Guoyin Wang, Keting Yin, Zhou Zhao, Hongxia Yang, Fan Wu, Shengyu Zhang, and Fei Wu. OS agents: A survey on MLLM-based agents for computer, phone and browser use. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7436–7465, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.369. URL https://aclanthology.org/2025.acl-long.369/.

Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. MobileWorld: Benchmarking autonomous mobile agents in agent-user interactive and MCP-augmented environments, 2025.

Guangyi Liu, Pengxiang Zhao, Yaozhen Liang, Liang Liu, Yaxuan Guo, Han Xiao, Weifeng Lin, Yuxiang Chai, Yue Han, Shuai Ren, Hao Wang, Xiaoyu Liang, Wenhao Wang, Tianze Wu, Zhengxi Lu, Siheng Chen, Linghao Li, Guanjing Xiong, Yong Liu, and Hongsheng Li. LLM-powered GUI agents in phone automation: Surveying progress and prospects. *Transactions on Machine Learning Research*, 2025. URL https://arxiv.org/abs/2504.19838.

Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for Pure Vision Based GUI Agent, 2024.

Mastodon gGmbH. Mastodon, 2024. URL https://joinmastodon.org.

Mattermost, Inc. Mattermost, 2024. URL https://mattermost.com.

Microsoft. Microsoft Copilot, 2023. URL https://copilot.microsoft.com.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=dHng2O0Jjr.

Qwen Team. Qwen3.5-27B, 2026. URL https://huggingface.co/Qwen/Qwen3.5-27B.

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=il5yUQsrjC.

Reddit. Reddit's AI Search, 2024. URL https://support.reddithelp.com/hc/en-us/articles/32026729424916-Reddit-s-AI-search.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2302.04761.

Grace Shao and Yaling Jiang. RedNote's AI Strategy: A Human-Centered Playbook, 2025. URL https://aiproem.substack.com/p/rednotes-ai-strategy-a-human-centered.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2303.17580.

Yueqi Song, Frank F. Xu, Shuyan Zhou, and Graham Neubig. Beyond Browsing: API-Based Web Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 11066–11085, Vienna, Austria, 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.findings-acl.577/.

South China Morning Post. China's Quora-like Zhihu Unveils AI-powered Search-and-Answer Feature to Engage More Users, 2024. URL https://www.scmp.com/tech/tech-trends/article/3268632/chinas-quora-zhihu-unveils-ai-powered-search-and-answer-feature-engage-more-users.

South China Morning Post. Tencent Adds AI Chatbot Friend to WeChat to Keep Users Glued to Super App, 2025. URL https://www.scmp.com/tech/big-tech/article/3303934/tencent-adds-ai-chatbot-friend-wechat-keep-users-glued-super-app.

StepFun AI. AndroidDaily: A verifiable benchmark for mobile GUI agents on real-world closed-source applications, 2025. URL https://huggingface.co/datasets/stepfun-ai/AndroidDaily/viewer/default/train.

Yifan Sui, Xin Huang, Hongbing Li, Fang Xu, Jiahe Lv, Haolong Yan, Yeqing Shen, Litao Liu, Zhimin Fan, Ziyang Meng, Jia Wang, Junbo Qi, Kaijun Tan, Zheng Ge, Xiangyu Zhang, Daxin Jiang, and Osamu Yoshie. AndroidDaily: A verifiable benchmark for mobile GUI agents on real-world closed-source applications, 2026.

Tongyi-MAI. MobileWorld: Benchmarking autonomous mobile agents in agent-user interactive and MCP-augmented environments, 2025a. URL https://huggingface.co/datasets/Tongyi-MAI/MobileWorld/viewer/default/train.

Tongyi-MAI. MobileWorld Leaderboard, 2025b. URL https://tongyi-mai.github.io/MobileWorld/#leaderboard.

TravelDaily. Qunar Launches "AI Xiaoluotuo": A ChatGPT-style In-App Travel Assistant for Itinerary Planning and Q&A, 2023. URL https://m.traveldaily.cn/article/171172.

Trip.com. Introducing TripGenie: A Ground-Breaking AI Travel Assistant by Trip.com for Unrivalled, Personalised and Intuitive Travel Planning and Booking, 2023. URL https://www.trip.com/newsroom/introducing-tripgenie-groundbreaking-ai-travel-assistant/.

Walmart. Walmart: The Future of Shopping Is Agentic. Meet Sparky, 2025. URL https://corporate.walmart.com/news/2025/06/06/walmart-the-future-of-shopping-is-agentic-meet-sparky.

Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception, 2024.

Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-Powered Task Automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, MobiCom '24, pp. 543–557, New York, NY, USA, 2024. Association for Computing Machinery. doi: 10.1145/3636534.3649379. URL https://doi.org/10.1145/3636534.3649379.

WPS Office. WPS AI, Reshape Your Workflow, 2023. URL https://www.wps.com/en-US/ai-homepage/.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Large language model-brained GUI agents: A survey, 2024a. URL https://arxiv.org/abs/2411.18279.

Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. UFO: A UI-Focused Agent for Windows OS Interaction, 2024b.

Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3706598.3713600. URL https://doi.org/10.1145/3706598.3713600.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 193–210, Santa Clara, CA, 2024. USENIX Association. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.